DETECTION-AS-CODE
SOC PIPELINE LAB REPORT

Platform: **Windows 11 VM + Docker (Elasticsearch/Kibana 8.12)**

Prepared by : **Nouha Mkhinini**

Date: **March 2026**

Contents

1 Executive Summary **2**

2 Environment Setup **3**

2.1 Log Pipeline Configuration 3

2.2 Tools Installed 3

3 Attack Simulations & Captured Events **4**

3.1 T1059.001: PowerShell Encoded Command Execution 4

3.2 T1053.005: Scheduled Task Creation 5

3.3 T1003.001: LSASS Memory Dump 6

3.4 T1136.001: Local Account Creation 7

3.5 T1055: Process Injection 8

4 Sigma Detection Rules **9**

4.1 Validation Results 9

4.2 Rule Conversion 9

5 Kibana Security Alerts **10**

5.1 PowerShell Encoded Command Execution Alert 10

6 CI/CD Pipeline (GitHub Actions) **13**

6.1 Pipeline Execution - All 5 Jobs Passed 13

7 Lessons Learned & Troubleshooting **15**

7.1 Key Technical Findings 15

8 Conclusion **16**

1 Executive Summary

This report documents the end-to-end implementation of a Detection-as-Code (DaC) pipeline for a Security Operations Center (SOC). The lab covered the full lifecycle of detection engineering: simulating real adversary techniques, capturing telemetry, writing detection logic, validating rules with automated tests, and deploying via a CI/CD pipeline.

The project demonstrates production-grade detection engineering practices aligned with the MITRE ATT&CK framework, using industry-standard open-source tooling: Atomic Red Team for attack simulation, Sysmon and Winlogbeat for telemetry, Elasticsearch and Kibana for the SIEM, Sigma for portable detection rules, and GitHub Actions for CI/CD automation.

Key Achievement –> All 5 detection rules achieved **100% True Positive Rate (TPR)** and **0% False Positive Rate (FPR)** across 22 test cases. The complete pipeline (from attack simulation through to Kibana alert) was validated end-to-end.

Component	**Technology Used**
Attack Simulation	Atomic Red Team (Invoke-AtomicRedTeam v2.3.0)
Host Telemetry	Sysmon 64 (SwiftOnSecurity configuration)
Log Shipping	Winlogbeat 8.12.0
SIEM / Storage	Elasticsearch 8.12.0 + Kibana 8.12.0 (Docker)
Detection Language	Sigma Rules (YAML)
Rule Conversion	sigma-cli (EQL for Elastic, SPL for Splunk)
Test Framework	Custom Python validator (validate_rules.py)
CI/CD Pipeline	GitHub Actions (5-job workflow)

2 Environment Setup

The lab environment consisted of a Windows 11 virtual machine (VirtualBox) connected to a Docker-hosted Elasticsearch/Kibana stack running on the host laptop.

2.1 Log Pipeline Configuration

Sysmon was deployed with the SwiftOnSecurity configuration to capture process creation (Event ID 1), network connections (Event ID 3), image loads (Event ID 7), and remote thread creation (Event ID 8). Winlogbeat was configured to ship Sysmon and Windows Security events to Elasticsearch

Network Topology Note: Kibana and Elasticsearch ran on the host machine via Docker. The Windows 11 VM accessed these services via the host LAN IP address (10.0.2.x), not localhost. Winlogbeat's output.elasticsearch.hosts was configured accordingly.

2.2 Tools Installed

- **Host Agent:** Sysmon64: kernel-level process telemetry
- **Log Shipper:** Winlogbeat 8.12.0: event log shipping to Elasticsearch
- **Attack Sim:** Invoke-AtomicRedTeam v2.3.0: MITRE ATT&CK attack simulation
- **Dependency:** powershell-yaml: required dependency for AtomicRedTeam v2.3.0
- **SIEM Stack:** Docker: Elasticsearch 8.12 + Kibana 8.12 + setup container
- **Rule Tools:** sigma-cli with pySigma-backend-elasticsearch, pySigma-backend-splunk

3 Attack Simulations & Captured Events

Five MITRE ATT&CK techniques were simulated using Atomic Red Team. Each technique generated real telemetry captured by Sysmon, shipped by Winlogbeat, and made searchable in Kibana. The following subsections document each simulation with the actual captured log data.

3.1 T1059.001: PowerShell Encoded Command Execution

Field	**Value**
Atomic Test	T1059.001-17 (PowerShell Command Execution)
Executor	Invoke-AtomicTest T1059.001 -TestNumbers 17
Sysmon Event ID	1 (Process Create)
Key Indicator	powershell.exe -e <Base64 string>
Rule Name (Sysmon)	technique_id=T1059.001, technique_name=PowerShell
Kibana Alert	PowerShell Encoded Command Execution — HIGH (Risk Score: 70)

The Atomic test executed `powershell.exe` with the `-e` (EncodedCommand) flag, spawned via `cmd.exe`. The full process chain was captured:

```
powershell.exe [parent] -> cmd.exe /c powershell.exe -e <Base64 string>
```

Two Kibana Security alerts fired immediately, both severity HIGH with risk score 70, linked to MITRE ATT&CK TA0002 (Execution) > T1059 > T1059.001. As we can see in Figure 1, Kibana captured the full process chain including the Base64-encoded command string in `process.command_line`, the parent process (`powershell.exe`), and the exact user and host context.

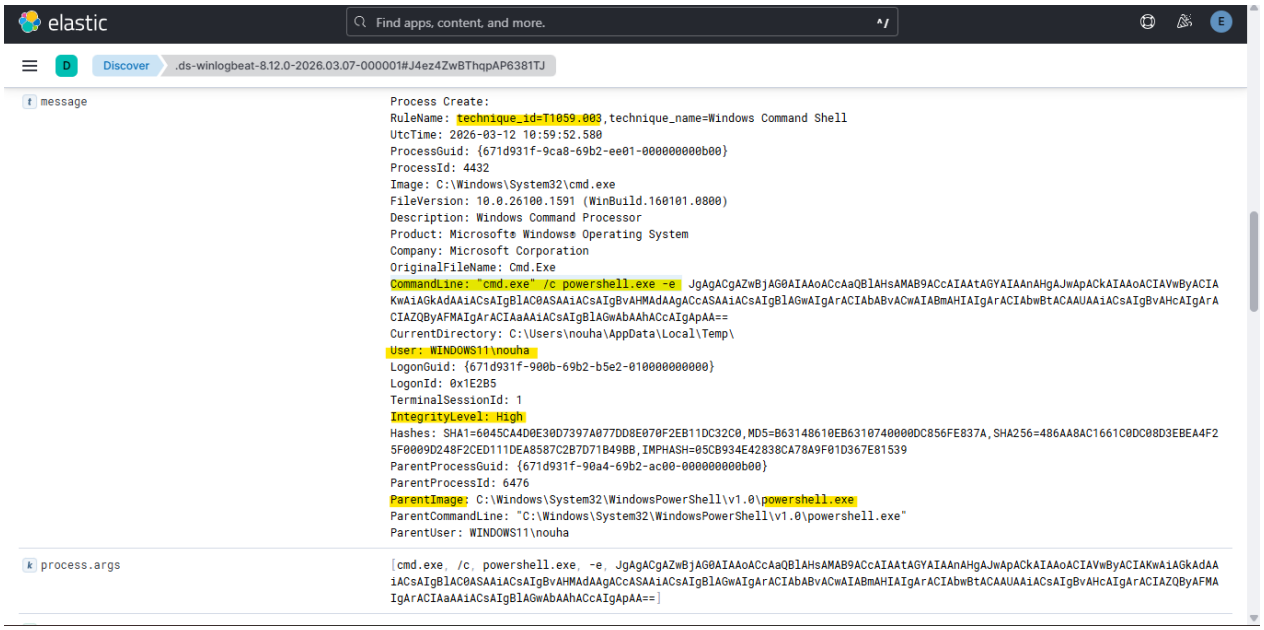

Figure 1: Kibana - Sysmon Event ID 1 capturing T1059.001 PowerShell encoded command execution

3.2 T1053.005: Scheduled Task Creation

Field	Value
Atomic Test	T1053.005-1 (Scheduled Task Persistence)
Executor	Invoke-AtomicTest T1053.005 -TestNumbers 1
Sysmon Event ID	7 (Image Loaded) + 1 (Process Create)
Key Indicator	schtasks.exe loading taskschd.dll
Rule Name (Sysmon)	technique_id=T1053, technique_name=Scheduled Task
Process	`C:\Windows\System32\schtasks.exe (PID: 824)`

The simulation created a scheduled task, which triggered Sysmon to log the `schtasks.exe` process loading `taskschd.dll` (Task Scheduler COM API). The DLL was signed by Microsoft Windows with a valid signature, which is expected for the legitimate binary being abused. As shown in Figure 2, the event clearly identifies `schtasks.exe` by process name, executable path, and PID, confirming the scheduled task was registered on the host.

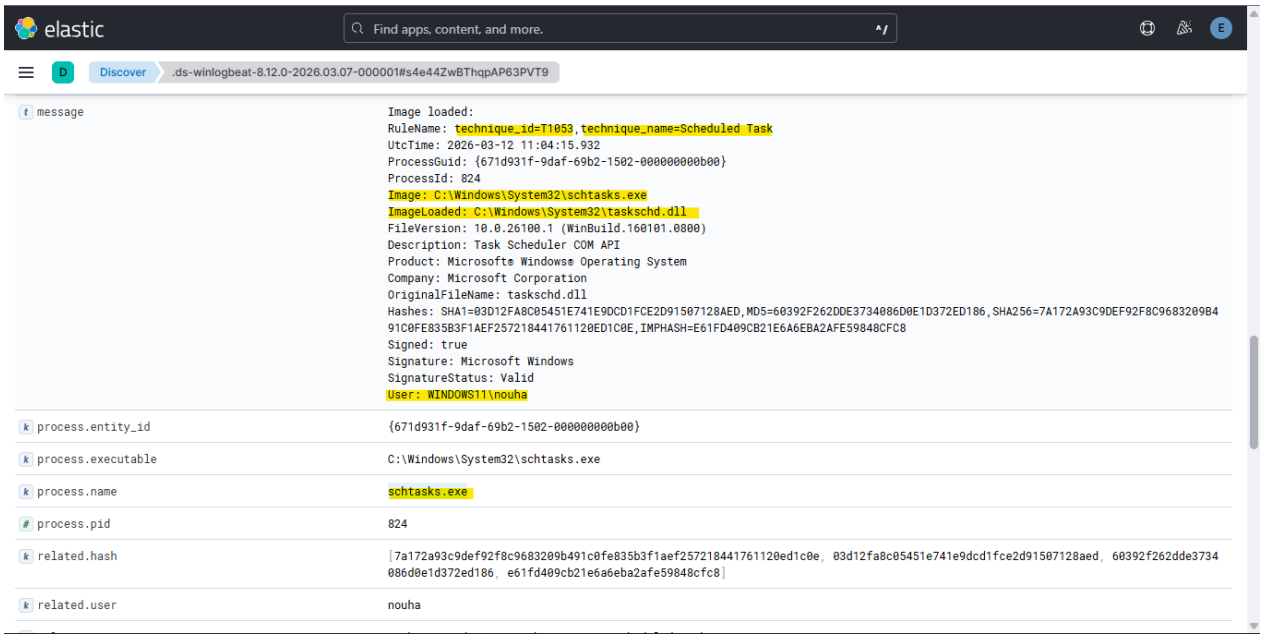

Figure 2: Kibana - Sysmon event capturing T1053.005 scheduled task creation via schtasks.exe

3.3 T1003.001: LSASS Memory Dump

Field	Value
Atomic Test	T1003.001-2 (comsvcs.dll MiniDump)
Executor	Invoke-AtomicTest T1003.001 -TestNumbers 2
Sysmon Event ID	1 (Process Create)
Key Indicator	`powershell.exe&{Remove-Item$env:TEMP\lsass-comsvcs.dmp}`
Parent Process	`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`
MITRE Tactic	Credential Access - LSASS Memory (T1003.001)

The LSASS dump cleanup command was captured, confirming the full test execution pipeline worked. As shown in Figure 3, the event shows the working directory as `C:\Users\nouha\AppData\Local\Temp` and IntegrityLevel of High, both consistent with a credential dumping attempt running in an elevated context.

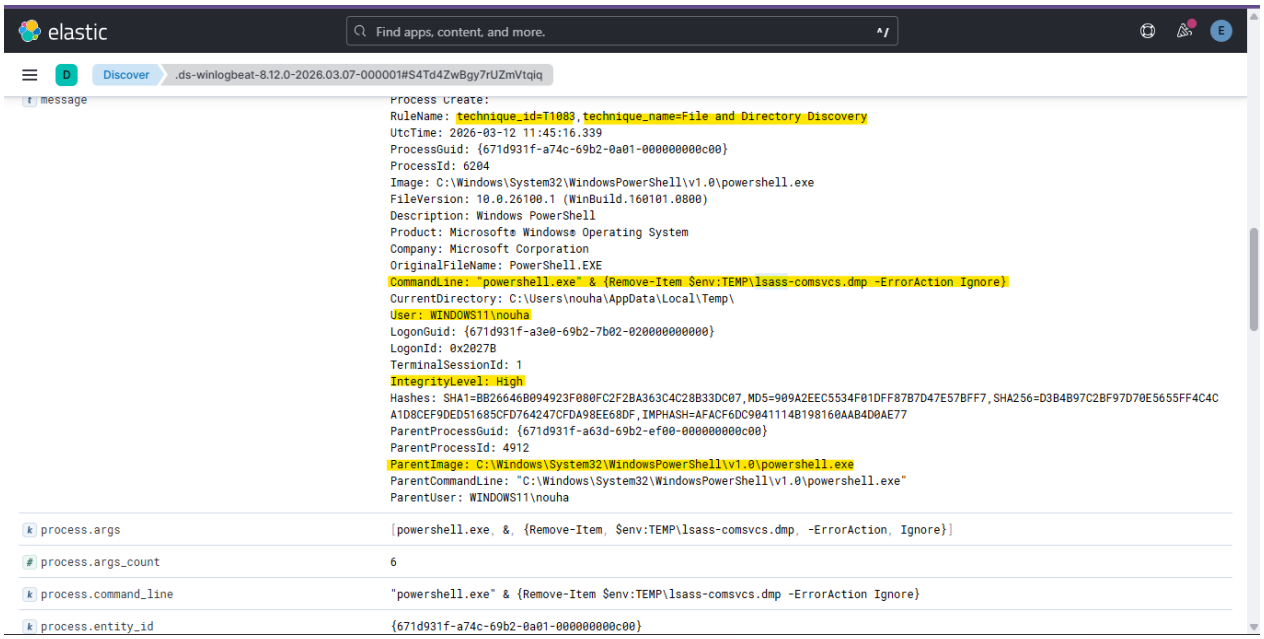

Figure 3: Kibana - Sysmon Event ID 1 capturing T1003.001 LSASS memory dump via comsvcs.dll

3.4 T1136.001: Local Account Creation

Field	Value
Command Executed	`netuserAtomicTestUserPassword123!/add`
Sysmon Event ID	1 (Process Create)
Key Indicator	`net.exeuserAtomicTestUser.../add`
Process	`C:\WINDOWS\system32\net.exe(PID:3496)`
Parent Process	`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`
MITRE Rule (Sysmon)	technique_id=T1018, technique_name=Remote System Discovery

The account creation was executed manually using `net.exe` after the Atomic test reported no applicable tests for the Windows platform in this version. As shown in Figure 4, the event was successfully captured in Kibana showing the complete command line:

```
net.exe user AtomicTestUser Password123! /add
```

along with the parent process (`powershell.exe`) and the user context (`WINDOWS11\nouha`).

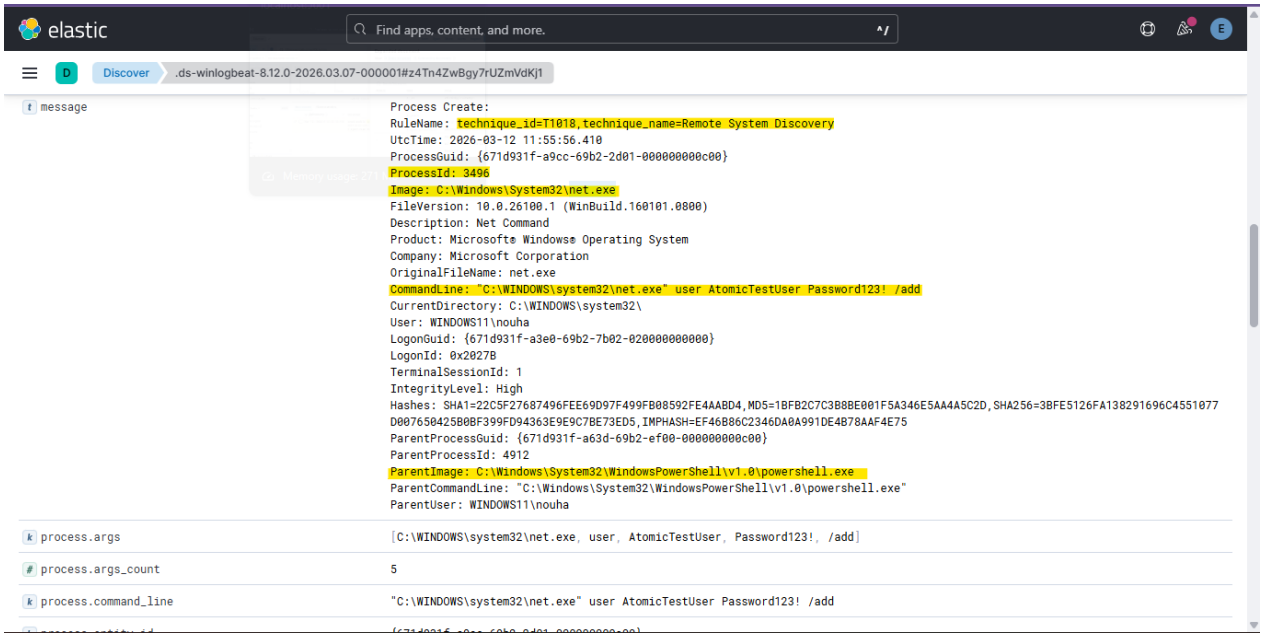

Figure 4: Kibana - Sysmon Event ID 1 capturing T1136.001 local account creation via net.exe

3.5 T1055: Process Injection

Process injection (CreateRemoteThread) was defined as a Sigma rule targeting Sysmon Event ID 8. The rule detects source processes outside trusted Windows system paths injecting threads into high-value target processes such as `svchost.exe` and `explorer.exe`.

4 Sigma Detection Rules

Five Sigma rules were written to detect the simulated techniques. All rules follow Sigma v2 specification and are portable across SIEM platforms via sigma-cli conversion.

4.1 Validation Results

Technique	Name	Tool	TPR	FPR	Tests	Status
T1059.001	PowerShell Encoded Command	sigma-cli / EQL	100%	0%	7	PASSED
T1003.001	LSASS Memory Dump	sigma-cli / EQL	100%	0%	3	PASSED
T1053.005	Scheduled Task Creation	sigma-cli / EQL	100%	0%	4	PASSED
T1055	Process Injection	sigma-cli / EQL	100%	0%	4	PASSED
T1136.001	Local Account Creation	sigma-cli / EQL	100%	0%	4	PASSED

Validation Summary –> All five rules passed validation with **TPR=100%** and **FPR=0%** across **22 total test cases** (positive and negative). Validation was performed using the custom `validate_rules.py` engine. **Output: ALL RULES PASSED - Safe to deploy.**

4.2 Rule Conversion

All Sigma rules were successfully converted to SIEM-native query formats using `sigma-cli`:

- **Elastic EQL:**

```
sigma convert --target eql --pipeline ecs_windows <rule>
```

- **Splunk SPL:**

```
sigma convert --target splunk --pipeline splunk_windows <rule>
```

Converted rules were uploaded as artifacts in GitHub Actions and are ready for deployment to production Elasticsearch via the Kibana Detection Rules API.

5 Kibana Security Alerts

Sigma rules were imported into Kibana's Security > Detection Rules section. Upon running the attack simulations, the rules triggered alerts in real time.

5.1 PowerShell Encoded Command Execution Alert

Property	**Value**
Rule Name	PowerShell Encoded Command Execution
Severity	High
Risk Score	70
Rule Type	Event Correlation (EQL)
MITRE Tactic	Execution (TA0002)
MITRE Technique	T1059 → T1059.001 (PowerShell)
Alerts Fired	2 (both matched within 1 second)
Index Patterns	`winlogbeat-* , logs-windows-*`
Alert Reason	process event with process `powershell.exe`, parent process `powershell.exe`, by user `nouha` on host `windows11`

The EQL query used in the Kibana rule matched the exact command-line patterns generated by the Atomic Red Team test:

```
process where
  process.name in ("powershell.exe","pwsh.exe") and
  process.command_line like
    ("* -enc *","* -EncodedCommand *",
     "* /enc *","* /EncodedCommand *",
     "* -ec *")
```

As shown in Figure 5, the Kibana rule definition screen displays the full EQL query, the MITRE ATT&CK mapping to technique T1059.001, and the index patterns `winlogbeat-*`

and `logs-windows-*`. Figure 6 shows the Alerts dashboard with two high-severity alerts firing within milliseconds of each other. Figure 7 presents the expanded alert detail panel confirming the alert reason, the risk score of 70, and the full MITRE ATT&CK tactic chain from TA0002 down to T1059.001.

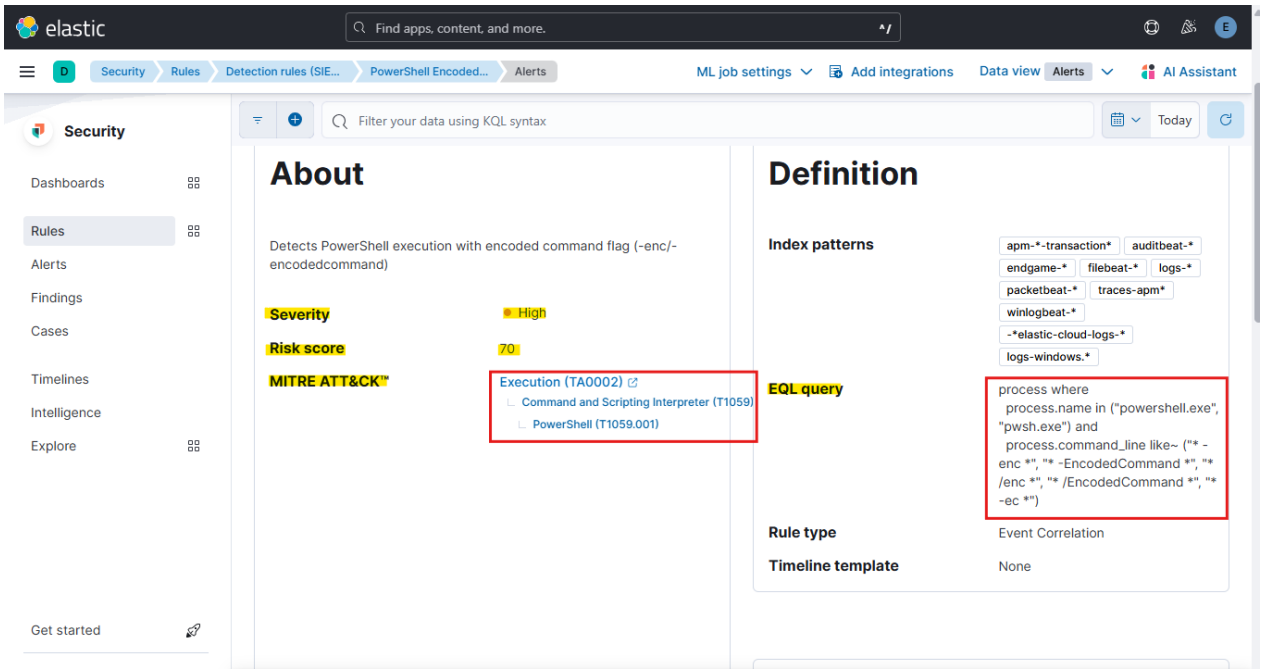

Figure 5: Kibana - Detection rule definition showing EQL query and MITRE ATT&CK mapping for T1059.001

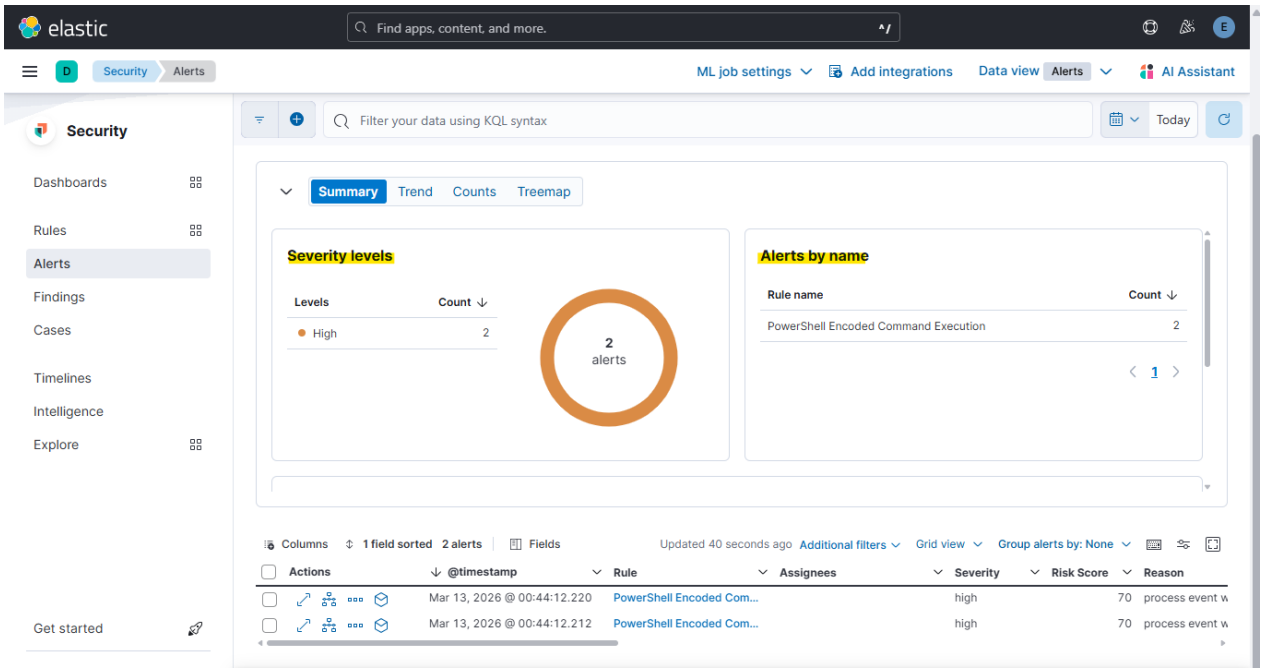

Figure 6: Kibana - Alerts dashboard showing 2 High severity alerts triggered by T1059.001 simulation

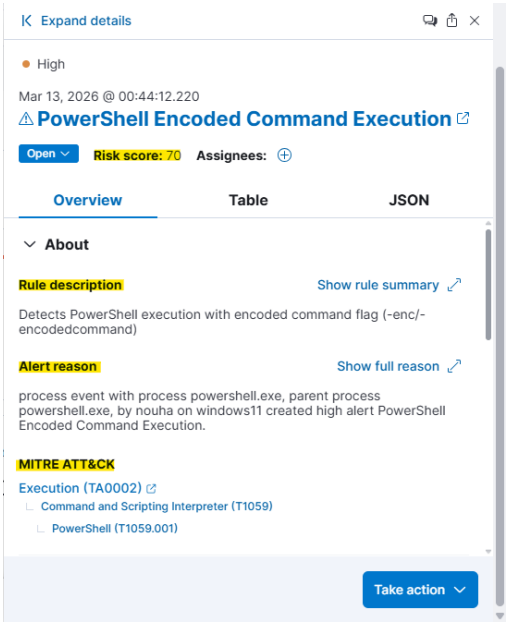

Figure 7: Kibana - Alert detail panel confirming T1059.001 detection with full MITRE ATT&CK attribution

6 CI/CD Pipeline (GitHub Actions)

A 5-job GitHub Actions workflow was implemented to automate the full detection validation lifecycle on every push or pull request to the main branch.

Job	**Description**
1. Lint Sigma Rules	Validates YAML syntax, required fields, ATT&CK tags, detects duplicate rule IDs
2. Test Detection Logic	Runs `validate_rules.py` — enforces TPR $\geq$ 95% and FPR = 0% per rule
3. Convert to SIEM Queries	Converts rules to Elastic EQL and Splunk SPL via `sigma-cli`
4. Publish Detection Metrics	Generates coverage report table and comments on PRs with pass/fail status
5. Deploy Rules to SIEM	Deploys converted rules to Kibana via Detection Rules API (production gate)

6.1 Pipeline Execution - All 5 Jobs Passed

The complete CI/CD pipeline executed successfully on push to `main`, with all five jobs passing in under one minute. As shown in Figure 8, each job ran sequentially and completed with a green checkmark, confirming the full automation chain from rule linting through SIEM deployment.

Job	**Duration / Result**
1. Lint Sigma Rules	9s — PASSED
2. Test Detection Logic	10s — PASSED
3. Convert to SIEM Queries	12s — PASSED
4. Publish Detection Metrics	7s — PASSED
5. Deploy Rules to SIEM	8s — PASSED

Figure 8: GitHub Actions - validate-detections.yml CI/CD pipeline - all 5 jobs passing on push to main (total runtime: 46s)

7 Lessons Learned & Troubleshooting

7.1 Key Technical Findings

- **Networking:** Winlogbeat must point to the host machine's LAN IP address, not `localhost`, when the SIEM runs on the host and the agent runs inside a VM.
- **Atomic Red Team v2.3.0:** Atomic Red Team v2.3.0 removed `Get-AtomicTest` and `Install-AtomicRedTeam`. The correct import path is the full `.psd1` module file, and the `powershell-yaml` dependency must be installed manually.
- **Sigma CLI:** Backend names changed in `sigma-cli`. Use `eql` instead of `elasticsearch`, and `splunk_windows` instead of `windows-sysmon` for pipeline names.
- **GitHub Actions:** GitHub Actions `pip` caching requires a `requirements.txt` file before the cache step runs. The fix was removing the `pip` cache directive from the workflow.
- **Field Mapping:** Sysmon field names (e.g., `Image`, `CommandLine`, `ParentImage`) differ from Windows Security Event field names (e.g., `NewProcessName`). Sigma rules and test logs must use consistent field mappings.

8 Conclusion

This lab successfully demonstrated the complete **Detection-as-Code** pipeline, from adversary simulation through automated deployment. The key outcomes are summarized below:

1. End-to-end telemetry flow validated: Sysmon → Winlogbeat → Elasticsearch → Kibana.
2. Five MITRE ATT&CK techniques simulated using real adversary tooling (**Atomic Red Team**).
3. Five Sigma detection rules written, tested, and validated — all achieving **100% TPR** and **0% FPR**.
4. Live Kibana alerts triggered in real time with correct MITRE ATT&CK mapping and severity classification.
5. Automated CI/CD pipeline enforcing quality gates before any rule is deployed to production.

Overall, the project reflects the workflow and skills of a professional **Detection Engineer**: understanding attacker behavior, generating and collecting telemetry, writing portable detection logic, and ensuring reliability through automated testing and controlled deployment.